

Implementação Prática da Tríade CIA na Segurança da Informação

Estudo de caso focado na aplicação de mecanismos criptográficos estruturados e algoritmos de resiliência corporativa.

INTEGRANTES DO GRUPO DE DESENVOLVIMENTO

Ana, Desiree, Giovana Marsigli, Isabelle, Nicole e Mariana Akemi

LINGUAGEM BASE

Python 3.x

DATA DE PUBLICAÇÃO

Junho de 2026

I 1. DESCRIÇÃO DA APLICAÇÃO

1.1 Objetivo Geral

Este documento técnico descreve a concepção e os fundamentos operacionais de um ecossistema desenvolvido especificamente para tangibilizar os três pilares que sustentam a Segurança da Informação: Confidencialidade, Integridade e Disponibilidade (Tríade CIA). O software simula de forma robusta e modularizada um barramento corporativo responsável pelo tráfego de dados altamente sensíveis, espelhando controles modernos de ambientes de produção em larga escala.

1.2 Abordagem dos Pilares Operacionais

A arquitetura foi estruturada para mitigar vulnerabilidades sistêmicas e canais laterais através de contramedidas programáticas fundamentadas em bibliotecas criptográficas consolidadas no mercado. Cada linha de código foi desenhada com base no princípio de *Security by Design*, garantindo que os mecanismos atuem de forma nativa e integrada:

Confidencialidade: Salvaguarda de payloads textuais e binários em repouso por meio de criptografia simétrica com disposição de padrões.

Integridade: Blindagem transacional através de assinaturas criptográficas parametrizadas por chaves e checagem em tempo fixo.

Disponibilidade: Controle perimetral de requisições, tolerância adaptativa a falhas temporárias e persistência redundante de estados históricos.

A seguir, o relatório esmiúça o detalhamento de engenharia aplicado a cada componente da aplicação, seus respectivos trechos lógicos de código e as justificativas para a adoção de tais tecnologias.

I PILAR I — CONFIDENCIALIDADE

Mecanismo Utilizado: Especificação Fernet (Cifragem Simétrica AES-128-CBC + Autenticação HMAC-SHA256).

Objetivo de Segurança: Proteção de strings estruturadas e volumes de arquivos em repouso contra acesso, inspeção ou exfiltração por entidades não autorizadas.

Explicação Técnica Detalhada

O pilar da Confidencialidade garante que a informação trafegue ou permaneça inacessível a qualquer agente desprovido de privilégios de descryptografia. No ecossistema implementado, essa proteção é estabelecida através da biblioteca

`cryptography`

sob a abstração `*Fernet*`. Internamente, o Fernet faz uso do algoritmo padrão global AES (Advanced Encryption Standard) com chaves de 128 bits operando sob o modo de encadeamento CBC (Cipher Block Chaining).

Diferente do modo pragmático ECB (Electronic Codebook) — que processa blocos de forma isolada e gera outputs idênticos para entradas repetidas —, o modo CBC insere um Vetor de Inicialização (IV) aleatório e realiza uma operação XOR entre o texto claro do bloco atual e o texto cifrado do bloco imediatamente anterior. Esse comportamento elimina por completo a análise estatística de padrões nos dados cifrados. Adicionalmente, o token Fernet realiza por padrão uma abordagem *Encrypt-then-MAC*, embutindo uma assinatura HMAC-SHA256 sobre o bloco cifrado e inserindo metadados de marcação temporal (timestamp). Isso impede a injeção de payloads modificados e ataques de repetição.

```
# Gera chave criptograficamente segura (256 bits via os.urandom)
chave = Fernet.generate_key()
cipher = Fernet(chave)

# Cifra dados sensíveis utilizando AES-128-CBC + HMAC-SHA256
dados_cifrados = cipher.encrypt(dados.encode('utf-8'))

# Descryptografia estrita: falha se a chave ou o token forem inválidos
dados_originais = cipher.decrypt(dados_cifrados).decode('utf-8')
```

I PILAR II — INTEGRIDADE

Mecanismo Utilizado: Funções Hash SHA-256 e HMAC-SHA256 Concorrentes.

Objetivo de Segurança: Detecção cirúrgica de modificações involuntárias ou adulterações maliciosas em trânsito, garantindo a autenticidade e não repúdio.

Explicação Técnica Detalhada

A integridade assegura que as informações não sejam modificadas de forma não autorizada sem que o sistema perceba a violação. O primeiro nível de defesa utiliza o SHA-256 (Secure Hash Algorithm), uma função hash matemática de via única que transforma dados de entrada de qualquer dimensão em um digest fixo de 256 bits. O algoritmo se apoia no *efeito avalanche*: a alteração de um único bit no payload de origem gera um hash completamente distinto.

No entanto, para cenários de comunicação activa, o hash simples é suscetível a ataques onde o invasor intercepta o canal, altera o arquivo e gera um novo hash legítimo correspondente à sua adulteração. Para neutralizar essa vulnerabilidade, o sistema integra o HMAC (Hash-based Message Authentication Code). O HMAC combina a função SHA-256 com uma chave secreta privada compartilhada unicamente entre instâncias confiáveis. Isso adiciona autenticação de origem ao processo: um atacante torna-se incapaz de forjar um hash válido sem possuir o segredo simétrico. A validação lógica das assinaturas em nossa aplicação é efetuada através da rotina

`hmac.compare_digest()`

, responsável por comparar as strings de bytes em tempo constante, blindando o sistema contra ataques colaterais de tempo (*timing attacks*).

```
# SHA-256: hash de 256 bits para verificação de integridade estrutural
hash_sha = hashlib.sha256(dados.encode('utf-8')).hexdigest()

# HMAC-SHA256: hash autenticado utilizando chave secreta compartilhada
hash_hmac = hmac.new(chave, dados.encode('utf-8'), hashlib.sha256).hexdigest() scrap

# Verificação em tempo constante para mitigação de timing attack
ok_hmac = hmac.compare_digest(hash_calculado, hash_esperado)
```

I PILAR III — DISPONIBILIDADE

Mecanismo Utilizado: Rate Limiting (Janela Deslizante), Backoff Exponencial e Backup com Timestamp.

Objetivo de Segurança: Garantia de continuidade operacional da API, resiliência contra indisponibilidade e blindagem contra exaustão de recursos.

Explicação Técnica Detalhada

A Disponibilidade garante que o sistema permaneça operacional e responsivo para transações legítimas. O ecossistema protege este pilar através de três abordagens complementares:

1. ***Rate Limiting via Janela Deslizante (Sliding Window):*** Ao contrário de algoritmos rígidos de janela fixa, a janela deslizante armazena em estruturas de dados dinâmicas os timestamps exatos de cada requisição realizada por usuário nos últimos segundos. Timestamps antigos que extrapolam o limite temporal configurado são expurgados em tempo real. Esse método mitiga abusos, ataques automatizados de força bruta e negações de serviço (DoS) de forma justa e precisa.
2. ***Retry Automático com Backoff Exponencial:*** Implementado por meio de decorators nativos, intercepta falhas decorrentes de instabilidades de rede ou serviços de terceiros. A política reexecuta o código estendendo o intervalo de espera de forma geométrica multiplicativa. Esse recuo escalonado impede o esgotamento precoce de tentativas e evita o efeito de "manada concorrente" sobre serviços instáveis.
3. ***Backup Automatizado Cronometrado:*** Cria cópias históricas idempotentes dos arquivos críticos anexando carimbos temporais estritos (timestamps) na classificação dos nomes. Esse controle atua como última linha de defesa em planos de recuperação de desastres (DRP) e resposta a incidentes de Ransomware.

```
# Rate Limiting: Janela deslizante thread-safe para controle de requisições
self.historico[usuario] = [t for t in self.historico[usuario] if agora - t <
self.janela_segundos]
if len(self.historico[usuario]) ≥ self.max_requisicoes:
    return False, 'Rate limit atingido'

# Retry com backoff exponencial aplicado via decorator abstrato
@retry_automatizado(max_tentativas=3, espera_base=1.0)
def operacao_instavel():
    pass

# Geração de backup automático estruturado com injeção de timestamp
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
shutil.copy2(origem, backup_dir / f"{nome}_{timestamp}.{ext}")
```

3. MATRIZ RESUMO DA TRÍADE CIA

PILAR CIA	MECANISMO ATIVO	ATIVO PROTEGIDO	ABORDAGEM OPERACIONAL
Confidencialidade	Fernet (AES-128-CBC)	Dados sensíveis em repouso	Cifra os dados com chave simétrica aleatória e impede leitura externa.
Integridade	SHA-256 + HMAC-SHA256	Payload de mensagens e arquivos	O hash detecta alterações locais e o HMAC valida a autenticidade da origem.
Disponibilidade	Rate Limiting / Backoff / Backup	Acesso contínuo e infraestrutura	Limita abusos volumétricos, reexecuta erros e mantém versionamento de dados.

4. CONCLUSÃO E LIÇÕES APRENDIDAS

4.1 Dificuldades Operacionais Encontradas

A principal barreira residiu em transpor os conceitos criptográficos abstratos para cenários lógicos reais, compreendendo as fraquezas estruturais de modos de operação obsoletos (como o modo ECB) frente ao encadeamento CBC. A acoplagem do HMAC em detrimento de abordagens de hash simples também demandou profunda análise matemática da equipe para mitigar ataques de extensão de comprimento.

No aspecto prático do código Python, a engenharia de um algoritmo de rate limiting totalmente thread-safe exigiu tratamento maduro sobre isolamento de memória e concorrência para evitar condições de corrida (race conditions). Além disso, a configuração precisa do backoff exponencial e o balanceamento entre o tempo de recuo do servidor e a tolerância do usuário final demandaram refinamento lógico constante.

4.2 Aprendizados Consolidados

A condução das atividades e testes práticos sedimentou lições críticas de engenharia de software seguro:

- **Insuficiência da Cifra Solitária:** Criptografia isolada oculta a informação, mas não atesta a legitimidade de sua origem; o HMAC é mandatório para provar o não repúdio e a integridade da origem.
- **Segurança Física do Tempo:** A implementação de validações através de funções em tempo constante é crucial para neutralizar ataques lógicos de canal lateral (*timing attacks*).
- **Disponibilidade Dinâmica:** Compreendeu-se que manter um sistema disponível vai além de garantir o estado ativo (Uptime); exige proteção contra abusos intencionais e fluxos de recuperação programados.
- **Abandono de Códigos Proprietários:** Entendeu-se a relevância de adotar pacotes e bibliotecas maduras e validadas pela comunidade internacional (como a cryptography) em substituição à criação manual de rotinas.

4.3 A Importância da Segurança no Ciclo de Vida do Software

A quebra de conformidade de qualquer pilar da tríade acarreta impactos destrutivos no mundo real. Violações de confidencialidade expõem dados privados à luz de legislações estritas; adulterações de integridade viabilizam fraudes financeiras sistêmicas; e indisponibilidades paralisam serviços críticos gerando prejuízos reputacionais e financeiros agudos.

Portanto, o desenvolvimento pautado em *Security by Design* consolidou-se nesta atividade não como uma funcionalidade cosmética de entrega, mas como uma responsabilidade técnica transversal e mandatória para a subsistência de qualquer software moderno. Cada linha de código pode se transformar em uma camada ativa de proteção ou em uma vulnerabilidade oculta exposta a atacantes. Desenvolver com essa mentalidade madura é uma competência indispensável para a atuação profissional em engenharia de tecnologia na atualidade.